Freight Rate Prediction

Approach, validation, final model, and required December chart

The task is supervised regression: predict each shipment's total posted rate in dollars. The final solution produces one positive rate for every required load ID and a controlled December scenario using the same trained estimator.

Final decision and evidence

The submitted model is core CatBoost with RMSE loss and 425 trees, fitted on all 48,000 labeled loads. It uses shipment and geographic features plus weekday; it excludes market_index and quote_signal. Its January-August-trained counterpart achieved MAE $110.39, RMSE $631.00, WAPE 4.61%, and signed bias $-24.27 on September-October.

The equipment-mileage baseline had MAE $229.10 on the same period: the core model reduced MAE by 51.8%. These are historical selection results, not known accuracy on the 12,000 final loads.

Disclosure: September-October was initially reserved for a later comparison but was already profiled during EDA. Its model results then changed the final selection from the market/MAE candidate to the core/RMSE candidate. It therefore contributed to model selection and must not be described as an independent final test. November-December outcomes remain hidden.

Data and quality checks

Dataset	Rows	Coverage / role
Labeled development	48,000	January-October 2025; posted_rate available
Final inference	12,000	November-December 2025; target unavailable
Prediction template	12,000	Required load IDs and blank predicted_rate
December scenarios	31	One fixed shipment; date changes daily

Dataset	Missing weights	Negative weights	Missing market index
Development	300	292	374
Final inference	165	145	249

Negative/nonpositive weights are replaced with missing values. Separate flags preserve whether weight was originally missing or invalid. CatBoost handles numerical missing values. No development or final rows were removed; no evaluation targets were clipped. The audit found no duplicate IDs or feature rows in the main files. Supplied coordinates were preserved rather than replaced with external geocoding.

Chronological validation and leakage prevention

The final task predicts two future months, so evaluation uses expanding training windows with two-month future periods. Calendar cutoffs keep all loads from a date on the same side. Random splitting would mix temporal regimes, especially the unstable quote signal.

Role	Training	Evaluation	Train rows	Eval rows
Fold A	Jan-Feb	Mar-Apr	9,255	9,855
Fold B	Jan-Apr	May-Jun	19,110	9,696
Fold C	Jan-Jun	Jul-Aug	28,806	9,671
Later comparison	Jan-Aug	Sep-Oct	38,477	9,523
Final refit	Jan-Oct	Nov-Dec (unlabeled)	48,000	12,000

For each CatBoost experiment, the final 14 calendar days inside the training window select the number of trees with early stopping (2,000-tree maximum; patience 100). The model is then refitted on the entire outer training partition. Outer evaluation labels are not used for early stopping. Scaling and fitted baseline statistics are learned inside each fitting partition. IDs and target-derived rate-per-mile diagnostics are excluded from model features.

MAE is the provisional primary selection metric; RMSE, WAPE, and signed bias reveal other behavior. The organizer's hidden ranking metric was not specified. WAPE is total absolute error divided by total actual rates; it is not an accuracy percentage. Pooled scores are recomputed from per-load out-of-fold predictions, not averaged RMSEs.

Recorded development experiments: March-August

Model	MAE ($)	RMSE ($)	WAPE (%)	Bias ($)
Market / MAE, scaled	115.27	602.42	4.80	-78.89
Market / MAE, scaled + quote	125.09	605.09	5.21	-65.16
Market CatBoost / RMSE	125.98	605.85	5.24	-58.65
Core CatBoost / RMSE [final recipe]	148.80	612.77	6.19	-89.65
Ridge	166.69	616.69	6.94	-92.20
Linear Huber	172.95	619.62	7.20	-115.28
Equipment x mileage	206.56	626.69	8.60	-32.86
Market / MAE, unscaled [underfit]	1,099.55	1,522.70	45.76	-414.47
Constant median	1,143.53	1,557.16	47.59	-414.27

These experiments share 29,222 evaluated loads. Adding market index improved early-period results. A GPU Ordered/Gradient MAE run on raw dollar targets severely underfit; scaling the target by 1,000 and reversing that scaling at inference corrected its optimization behavior. This target scaling belongs to the experimental MAE models; the final RMSE model predicts dollars directly.

Adding quote_signal worsened pooled MAE and two of three folds. Its relationship with rate per mile changes sign by month and is near zero in August. It was excluded for observed instability. The data does not establish its provenance, so the report does not assert proven target leakage.

Later-period selection and generalization

January-August-trained model	MAE ($)	RMSE ($)	WAPE (%)	Bias ($)
main_mae_market	168.47	651.44	7.04	-145.16
chart_core_rmse	110.39	631.00	4.61	-24.27
equipment_mileage	229.10	670.66	9.58	40.93

The market/MAE candidate led on earlier folds but degraded in September-October. The core RMSE model was selected for stronger performance in this most recent period and for a single consistent prediction path across both required input formats. This is a recency-based engineering decision; it does not establish that core RMSE is universally best. No further parameter search was performed after this decision.

Unfamiliar-city stress test: scope and limitation

There are 1,447 final loads involving a development-unseen city (12.06%) and 1,461 loads on unseen directional lanes. A supplementary fold-C stress test withheld four cities and all touching loads from its fitting data, then evaluated later loads involving those cities. The cities were chosen with a fixed seed without consulting their rates.

Stress-test model	Loads	MAE ($)	RMSE ($)
mileage_reduced_training	1465	328.22	907.88
catboost_reduced_training	1465	144.38	856.23
catboost_original_training_reference	1465	141.49	856.93

Important scope distinction: this stress test evaluated the market/MAE candidate, not the final core RMSE model. Its exact scores cannot be claimed as the final model's unseen-city performance. Removing cities also reduces training size, so the comparison does not isolate category novelty alone. The final core model includes geographic features and supports unseen categories, but its unseen-city accuracy remains a limitation requiring further validation in a production setting.

Alternative methods and stopping decision

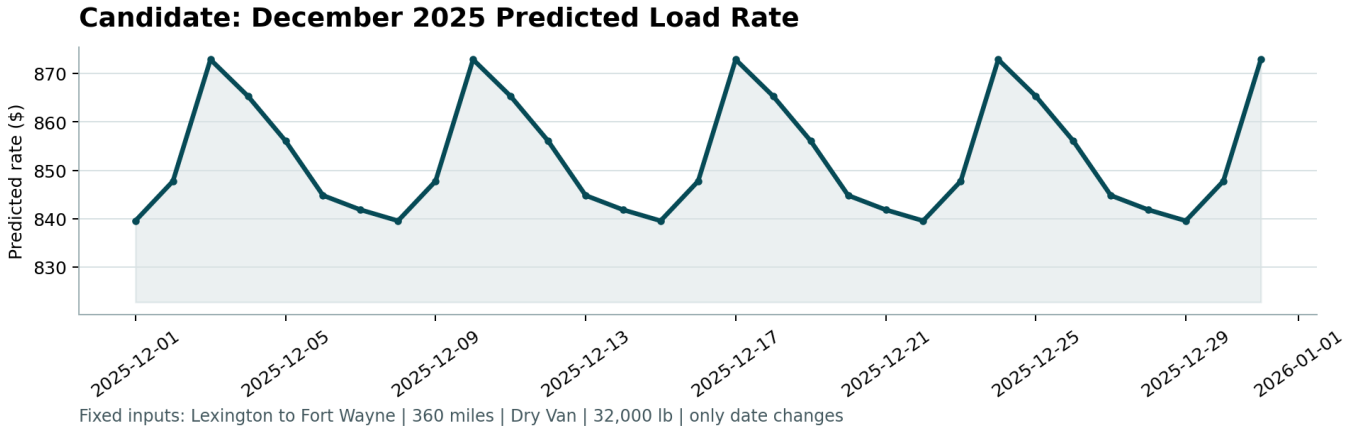
Constant pricing, equipment-mileage, Ridge, linear Huber, multiple CatBoost feature sets, and RMSE versus MAE objectives were evaluated. LightGBM, XGBoost, random forests, neural networks, and aggregate time-series methods were considered but not trained. They are alternatives, not failed experiments. Model search was bounded to avoid adding complexity without evidence. An additional multi-seed study was not performed; GPU training is not bitwise deterministic.

Final model and fixed December chart

The final estimator uses 425 trees, depth 6, learning rate 0.05, RMSE loss, and seed 42. The last 14 labeled days (October 18-31) selected its tree budget before refitting on all January-October rows. The final tree count differs from the earlier-period model because its inner training window differs.

Features: pickup and delivery city, equipment, four supplied coordinates, distance, cleaned weight, original-missing and invalid-weight flags, and weekday. No market index, quote signal, raw ID, target-derived feature, full-date trend, or holiday feature is used. No target scaling is applied to the final RMSE model.

The December file lacks coordinates. They are reconstructed from the consistent development-only city mapping; its supplied 360-mile distance remains unchanged. The same saved final model predicts both the 12,000 final loads and these scenarios.



This is the chart generated by the original score.py. Fixed inputs are Lexington to Fort Wayne, 360 miles, Dry Van, and 32,000 lb, for December 1-31, 2025. Only weekday changes in the model's feature representation. Identical weekdays therefore repeat the same prediction, with at most seven distinct input combinations.

The weekly repetition is expected model behavior, not a plotting defect. The model cannot represent a December-wide trend or a Christmas-specific surcharge without corresponding features and evidence. The chart is a controlled scenario, not a validated daily market forecast. There are no labeled December observations.

Submission verification and practical limitations

The packaged standalone script reloads the final model and reconstructs predictions for every final load and December scenario. Its predictions are checked against the submitted CSVs to a $0.000001 tolerance, allowing six-decimal CSV rounding. The official scorer then validates exactly 12,000 required unique IDs, column order, finite positive rates, and the 31 fixed December rows, and regenerates the required chart.

Scorer success establishes file/scenario validity, not predictive accuracy. November-December labels remain unavailable. The final model's accuracy on those 12,000 loads is unknown until the organizer evaluates it.

Large individual errors remain: RMSE is much greater than MAE. The earlier equipment-mileage diagnostic found roughly 1% of evaluated loads accounted for about 88% of squared error; that concentration statistic belongs to that baseline and was not measured for the final model. High-rate labels were retained. Tail errors, potential temporal drift, semantic accuracy of supplied coordinates, and weekday-only seasonality limit business deployment claims.

Reproduction

The archive includes the executed notebook, solution.py, pinned runtime dependencies, original input CSVs, unchanged score.py, recorded experiments, final model, model configuration, predictions, and this report. The notebook records the full exploration and model-development history. The standalone script supports final inference, saved-model verification, refitting, and reconstruction of the core model's later-period evaluation.

```
python -m pip install -r requirements.txt

python solution.py --mode verify

python solution.py --mode predict

python score.py --predictions validation_predictions.csv --december-predictions outputs/december_predictions.csv --output-dir scorer_results

python solution.py --mode train --task-type GPU

python solution.py --mode evaluate --task-type GPU
```

Prediction from the saved model does not require a GPU. Refitting uses the recorded final tree count by default; --select-iterations repeats the inner chronological stopping procedure. GPU training can vary slightly between runs, and changing training hardware can change defaults and results. Archived metrics describe the original recorded runs.